

PHASE 2.5A - AUTHENTICATION FRONTEND

Binding End-to-End Implementation Prompt for Claude Code

Project state	Phase 0.5, 1, 1.5 and 2 complete
Stack	React + TypeScript frontend, .NET 10 / ASP.NET Core backend, PostgreSQL
Objective	Implement authentication UX end-to-end before CRM Opportunity frontend
Primary onboarding	Enterprise invite / tenant-admin onboarding
Public registration	Supported only as a configurable capability; disabled by default unless current product config enables it
Non-negotiable	Reuse existing Identity / Access foundations; do not invent a parallel auth or permission system

Purpose

This document is intended to be given directly to Claude Code. It is deliberately explicit so that the implementation is repository-driven, security-first, and does not make product or architecture decisions silently. Claude must inspect the actual code first, create a concrete implementation plan, then implement end-to-end unless a truly blocking open decision is found.

The implementation phase is not a redesign of Phase 1.5 Access/Authorization. Authentication establishes identity and session; Phase 1.5 continues to own authorization and policy evaluation.

MASTER IMPLEMENTATION PROMPT

Copy/give the following specification to Claude Code as the binding task definition.

ROLE AND SESSION SCOPE

You are acting as a Principal Frontend Architect, Senior .NET 10 Engineer, Application Security Engineer, and Enterprise UX Engineer. This session is dedicated only to PHASE 2.5A - Authentication Frontend Integration and End-to-End Validation.

Completed phases:

- Phase 0.5 - MasterData / Party Foundation ☐
- Phase 1 - CRM Lifecycle / Pipeline Foundation ☐
- Phase 1.5 - Enterprise Access / Authorization Foundation ☐
- Phase 2 - CRM Opportunity Commands & API ☐

Current phase:

- Phase 2.5A - Authentication Frontend + End-to-End Identity/Session Flow

FOUNDATION-FIRST, NOT MVP-FIRST. Do not create temporary auth shortcuts that will have to be removed when enterprise SSO, multi-tenant onboarding, field security, or policy obligations are expanded later.

0. FIRST RULE - INSPECT BEFORE CHANGING

Do not start coding immediately. Inspect the actual repository and determine what authentication, identity, session, tenant-context, authorization, frontend-shell, API-client, and error-handling infrastructure already exists.

- Inspect the React application structure, router, shell, state/query libraries, API client, environment configuration and design system.
- Inspect .NET 10 Host startup, Identity/Auth modules, Access module, PrincipalResolver, tenant resolver, middleware, cookies/tokens, CORS/CSRF and current-user endpoints.
- Inspect Phase 1.5 authorization contracts and Phase 2 API conventions. Authentication must feed the same Principal/Tenant context they expect.
- Inspect tests and architecture docs. Actual code is the source of truth for implemented behavior; binding architecture documents are the source of truth for intended behavior.
- Do not trust filenames, old plans, or comments without verifying code paths.

1. TARGET USER EXPERIENCE

At the end of Phase 2.5A a real user must be able to enter the application, establish a valid authenticated session, resolve tenant context, reach protected application routes, manage password flows, and leave the application cleanly.

```
App start
↓
Resolve existing session
├─ no session → Login / Register / Invite / Reset flows
└─ valid session
    ↓
Current Principal
    ↓
Tenant membership / active tenant resolution
    ↓
Authorization bootstrap required by the shell
    ↓
Protected Application Shell
```

2. REQUIRED USER FLOWS

Implement and verify these flows end-to-end, adapting route names to the repository conventions rather than blindly copying examples:

- Login
- Logout
- Invite acceptance / first-time activation
- Registration entry flow
- Forgot password
- Reset password
- Change password
- Session restore after refresh
- Session expiry handling
- Unauthorized (401) handling
- Forbidden (403) handling
- Current user/principal bootstrap
- Tenant membership resolution
- Tenant selection when the user belongs to multiple tenants
- Protected routes and safe post-login redirect

3. REGISTRATION / ONBOARDING MODEL - BINDING

The enterprise default is invite-based or tenant-admin-created onboarding. Public self-registration is a platform capability, not the mandatory enterprise default.

Implement registration in a way that supports both models without duplicating identity logic. Public self-registration must be feature/configuration gated and disabled by default unless the existing product configuration explicitly enables it. Invite acceptance is the primary enterprise onboarding path.

A public registration path must never allow the browser to assign itself an arbitrary TenantId, role, scope, organization unit, or privileged capability. Tenant membership must be created by approved server-side onboarding rules.

4. AUTHENTICATION VS AUTHORIZATION - DO NOT MIX

Authentication answers: WHO is this?

Authorization answers: WHAT may this principal do?

Phase 2.5A may implement identity/session UX and the minimum backend endpoints needed for it. It must not redesign, duplicate, or bypass Phase 1.5 Access/Authorization.

After authentication, the established principal and tenant context must flow into Phase 1.5 exactly through the existing contracts. Never create a frontend-only permission model or a second CRM-specific authorization mechanism.

5. IDENTITY IMPLEMENTATION DECISION

First discover whether the repository already has an identity provider/session mechanism. Reuse it if it is architecturally valid.

If no local identity implementation exists, do not invent proprietary password hashing, token formats, or credential storage. Use a standard .NET identity mechanism suitable for local credentials and future federation. Prefer ASP.NET Core Identity or the already-selected external OIDC provider, based on repository evidence.

If choosing an identity provider would contradict a binding architecture decision or there are multiple incompatible existing implementations, mark OPEN DECISION and stop before

6. SESSION / TOKEN SECURITY

Follow the session model already established in the repository. If no model exists and local web authentication is required, prefer a secure server-managed session / HttpOnly cookie approach for the browser rather than storing bearer access tokens in localStorage.

- HttpOnly for session cookies where applicable.
- Secure in production.
- Appropriate SameSite policy.
- CSRF protection for cookie-authenticated state-changing requests.
- No passwords, refresh tokens, access tokens, reset tokens or invite tokens in logs.
- Do not persist sensitive authentication tokens in localStorage/sessionStorage unless an existing binding design explicitly requires it.
- Safe returnUrl validation; prevent open redirects.
- Logout must invalidate/terminate the server-side session or authentication cookie, not merely clear React state.

7. LOGIN

Implement Login UI and API integration with server-authoritative validation. Client validation is only for UX. The server owns credential validation, lockout/rate-limit behavior, membership checks and session issuance.

Login success must bootstrap the authenticated principal and resolve tenant context before protected business screens are rendered.

Login failure must not reveal unnecessary account security details. Reuse the backend error taxonomy and localization conventions.

8. INVITE ACCEPTANCE / FIRST-TIME USER

The invite flow must be treated as a signed/opaque server-controlled onboarding capability. The browser must not manufacture membership data.

```
Invite link
↓
Server validates token + expiry + intended identity + tenant
↓
User sets/establishes credentials if required
↓
Membership activated
↓
Session established or redirect to login
↓
Tenant/application shell
```

Do not implement tenant-admin user-management screens unless they already belong to the current scope. Phase 2.5A requires the invite acceptance path, not a full IAM administration console.

9. PUBLIC REGISTRATION

When public registration is enabled, the endpoint and UI must follow an explicit server-side onboarding policy. Registration must not automatically grant enterprise tenant privileges.

If self-registration is disabled, the registration route may show the configured invite-only/onboarding state instead of exposing an unusable form.

10. FORGOT / RESET PASSWORD

Implement the full request-to-reset flow without account enumeration.

```
Forgot Password
↓
Submit identifier
↓
Always return neutral response
↓
If account is eligible, server creates short-lived single-use reset token
↓
Email delivery abstraction
↓
Reset Password page
↓
Token validation + new password
↓
Success → login/session policy
```

- Do not expose whether the account exists in the forgot-password response.
- Reset tokens must be short-lived and single-purpose.
- Do not choose a production email vendor if the repository has no vendor decision. Reuse an existing email abstraction; otherwise create the minimum abstraction/dev sink needed and document the production provider as deferred.
- Do not log reset links or tokens in production logs. Development-only delivery/sink behavior must be clearly separated.

11. CHANGE PASSWORD

Authenticated users must be able to change their password through an account/security surface. Re-authentication/current-password requirements must follow the chosen identity provider semantics.

Analyze whether password change invalidates other sessions. Use existing policy if defined; otherwise mark the session-revocation behavior as an explicit decision in the implementation report rather than silently inventing it.

12. PASSWORD POLICY

Do not hard-code a separate password policy in React. The backend/identity provider is authoritative. The UI may render hints provided by configuration or shared validation metadata, but it must not drift from server rules.

Do not implement custom password cryptography. Use the selected identity provider/framework.

13. SESSION RESTORE AND EXPIRY

On browser refresh, the application must resolve the current session before deciding whether a protected route is accessible. Avoid UI flashes where protected content renders before authentication state is known.

```
Auth state examples:
- unknown/loading
- unauthenticated
- authenticated but tenant unresolved
- authenticated + tenant resolved
- session expired
```

When a session expires during use, central API handling should transition the application to a safe unauthenticated state and redirect to Login with a safe return target where appropriate. Do not create redirect loops.

14. TENANT RESOLUTION - SECURITY CRITICAL

A signed-in identity may belong to one or more tenants. The active tenant must be resolved through server-authoritative membership data.

1 membership → select automatically if current architecture allows
N memberships → tenant selector
0 memberships → onboarding / no-access state

The frontend must never gain access to a tenant merely by changing a TenantId value, route parameter, local storage value, or request header. The backend must validate active tenant membership on every relevant request using the existing tenant and authorization foundation.

Use the current project convention for carrying active tenant context. Do not invent a new header/session mechanism if one already exists.

15. CURRENT USER / AUTH BOOTSTRAP CONTRACT

Determine the minimum authoritative bootstrap data required by the application shell. Do not download the entire authorization graph into the browser.

Typical concepts may include identity display information, active tenant, memberships needed for tenant selection, and coarse shell/application capabilities. Resource-level, record-level, field-level and obligation decisions must still be evaluated server-side when the resource is used.

16. PROTECTED ROUTES

The React router must have a single consistent protection strategy. Route protection is UX/navigation control, not the security boundary. Backend APIs remain authoritative.

- Unauthenticated → Login.
- Authenticated but tenant unresolved → tenant selection/onboarding.
- Authenticated with tenant → protected application shell.
- Forbidden business route → dedicated 403/no-access experience without leaking hidden data.

17. API CLIENT AND ERROR HANDLING

Centralize auth-aware API behavior using the project's existing query/client layer (for example TanStack Query + shared fetch/HTTP client if already present). Do not scatter authentication retry/redirect logic through individual screens.

Handle at least: 400 validation, 401 unauthenticated/session expired, 403 forbidden, 404 tenant-safe not found, 409 concurrency/domain conflict where relevant, 429 throttled, 5xx service error.

18. 401 VS 403

401 = no valid authenticated session / credentials
403 = authenticated principal exists but is not permitted

Preserve this distinction throughout API client, router behavior and UI. Do not redirect every 403 to Login.

19. ACCESS / FIELD SECURITY / OBLIGATIONS

Phase 2.5A does not duplicate the detailed Opportunity UI yet, but the auth shell must be compatible with Phase 1.5 concepts: coarse authorization, tenant-safe load, record policy, field READ/WRITE security, and policy obligations/approval requirements.

Do not implement a browser-side global permission cache that becomes the source of truth. The frontend may use backend-provided capabilities for UX, but every protected command/query remains server-authoritative.

20. FRONTEND UI REQUIREMENTS

Use the existing design system and application shell conventions. Do not introduce a second component library without evidence that the repository requires it.

- Accessible labels and keyboard navigation.
- Clear loading, submitting, success and failure states.
- Prevent duplicate submissions while a mutation is in flight.
- Password fields support appropriate autocomplete attributes.
- No sensitive data in client console logs.
- Responsive layouts for common desktop/tablet widths; mobile-friendly auth screens where practical.
- Forms must preserve user-entered non-sensitive values on recoverable validation failures.

21. REACT STATE MANAGEMENT

Do not create a large client-side auth state machine if the server/session and query cache already provide the source of truth. Keep state ownership minimal and explicit.

Use TanStack Query or the established query library for server state. Use local state only for transient form/UI state. If Zustand or another store already owns shell session metadata, reuse it rather than introducing a new store.

22. BACKEND ENDPOINTS - INSPECT, THEN COMPLETE MINIMUM

List the existing identity/session endpoints before adding anything. If the frontend flow requires missing endpoints, implement the smallest correct backend surface in the appropriate Identity/Auth boundary, not inside CRM.

Conceptual endpoints may include login, logout, me/current-session, tenant memberships/select active tenant, invite validation/acceptance, register if enabled, forgot/reset password, change password. Names/routes are not binding; follow repository conventions.

23. RATE LIMITING / BRUTE-FORCE PROTECTION

Inspect current ASP.NET rate limiting, identity lockout, reverse-proxy/WAF controls and authentication provider settings. Authentication endpoints are abuse-sensitive.

Do not invent arbitrary thresholds if the project already has policy. If none exists, implement framework-supported baseline protection appropriate for development/production and document configurable thresholds rather than burying magic values in UI code.

24. AUDIT / EVIDENCE

Authentication/security events must use the project's audit/evidence strategy where applicable. Do not mix ordinary diagnostic logs with business/security evidence.

Analyze and preserve evidence for security-relevant events such as password change, invite acceptance, tenant switch, repeated failed login/lockout if the existing evidence policy requires it. Never write raw passwords/tokens into evidence or logs.

25. OBSERVABILITY

Use existing OpenTelemetry/logging conventions. Add correlation where needed. Authentication failures should be diagnosable without exposing secrets or creating account-enumeration telemetry visible to clients.

26. EMAIL / LINK URL GENERATION

Invite/reset links must use environment-aware trusted application base URLs. Do not build links from untrusted request Host headers without the project's approved host validation/proxy configuration.

All tokens in URLs must be opaque/signed/provider-generated and short-lived according to their purpose.

27. CORS / CSRF / SAME-ORIGIN

Inspect how React is deployed relative to .NET. Do not loosen CORS globally for convenience. If cookie authentication is used cross-origin during development, configure credentials, allowed origins, SameSite and CSRF deliberately. Production configuration should prefer the simplest secure origin model supported by deployment architecture.

28. TEST STRATEGY - REQUIRED

Add/extend tests at the appropriate layers. Do not consider the phase complete because the pages render.

- Backend unit/integration: login/session, tenant membership validation, reset/invite token behavior, change password, security failure paths.
- Use real PostgreSQL/Testcontainers when existing identity persistence or tenant behavior requires database semantics.
- API tests: 401/403 distinction, neutral forgot-password response, invalid/expired invite/reset token, logout invalidation, tenant switch authorization.
- Frontend component tests for form states where the repository has an established framework.
- Playwright E2E: login, logout, session restore, wrong credentials, forgot/reset (using safe test mail sink), invite acceptance, password change, protected route redirect, multi-tenant selection if test fixtures support it.
- E2E must verify that a browser cannot obtain access to a tenant merely by manipulating client state/request tenant identifiers.

29. SECURITY NEGATIVE TESTS

- Open redirect through returnUrl must fail.
- Expired/reset/invite token replay must fail as expected.
- Cross-tenant active-tenant manipulation must fail.
- Unauthenticated protected API call returns the correct authentication response.
- Authenticated but unauthorized call returns the established forbidden/not-found behavior without leaking hidden records.
- Password/reset/invite tokens never appear in normal logs or API responses beyond the intended delivery flow.

30. EXPLICIT NON-GOALS

Do not expand this phase into the CRM frontend or a full IAM suite. Unless already necessary for the auth flow, do not implement:

- Opportunity list/detail/create UI
- Customer Need Intelligence
- Quote/Sales UI
- Activity timeline
- Feedback
- KVKK/Consent management UI
- Full tenant user administration console
- Role/permission editor
- Enterprise SAML/OIDC federation UI

- SCIM provisioning UI
- MFA implementation unless the identity architecture already requires it in this phase
- AI/RAG/agent features

31. FILE / MODULE BOUNDARIES

Authentication/identity code belongs in the appropriate Identity/Auth boundary. Authorization policies remain in Access. CRM must not own authentication. React auth pages belong in the shared application/shell/auth area, not inside the CRM feature folder unless the existing frontend architecture explicitly requires otherwise.

32. IMPLEMENTATION WORKFLOW - DO NOT PAUSE UNNECESSARILY

Perform the work in this sequence:

```
Step 1 - Repository inspection
Step 2 - Write concrete Phase 2.5A implementation plan
Step 3 - Identify blocking open decisions
Step 4 - If no blocking decision exists, implement backend gaps + frontend flows
Step 5 - Add tests
Step 6 - Run formatting/build/tests/E2E
Step 7 - Perform security sanity review
Step 8 - Produce final implementation report
```

Do NOT stop after the plan merely to ask for permission. This prompt authorizes implementation. Stop only if a genuinely unresolved product/architecture decision would force you to invent a binding behavior.

33. REQUIRED PLAN DOCUMENT

Before code changes, create:

```
docs/architecture-analysis/PHASE_2_5A_AUTH_FRONTEND_IMPLEMENTATION_PLAN.md
```

The plan must include: verified current state; current identity/session architecture; frontend router/shell state; target flows; registration/invite behavior; tenant resolution; required backend endpoints; required frontend routes/components; API client changes; security controls; test matrix; exact files to add/change; migration impact; open decisions; implementation order.

34. OPEN DECISION PROTOCOL

If a binding decision is missing, do not silently invent it. Mark OPEN DECISION with: issue, repository evidence, options, security/architecture impact, migration impact, recommendation, and whether implementation is blocked.

Examples of decisions that may genuinely block implementation: incompatible identity providers already present; undefined credential ownership; contradictory tenant-session semantics; binding architecture requiring external IdP but no provider choice.

Do not treat ordinary implementation details as open decisions merely to avoid doing the work.

35. REQUIRED FINAL IMPLEMENTATION REPORT

After implementation create:

```
docs/architecture-analysis/PHASE_2_5A_AUTH_FRONTEND_FINAL_REPORT.md
```

Include: what existed before; what changed; routes/screens; API endpoints; identity/session model; tenant selection behavior; registration/invite behavior; security decisions; tests added; test/build results; migrations/config changes; deferred items; known risks; exact manual verification steps.

36. DEFINITION OF DONE

Phase 2.5A is complete only when all applicable items below are true:

- A user can authenticate from the React UI against the real .NET backend.
- A valid session survives browser refresh according to the selected session policy.
- Logout invalidates the real session and protected APIs are no longer accessible.
- Invite acceptance works end-to-end for enterprise onboarding.
- Public registration behavior is configuration-controlled and cannot self-grant tenant privileges.
- Forgot/reset password works end-to-end without account enumeration.
- Authenticated password change works.
- Session expiry produces a clean UI transition and safe redirect behavior.
- One-tenant and multi-tenant membership states are handled using server-authoritative membership.
- Protected routes are gated in UI while backend remains authoritative.
- 401 and 403 are handled distinctly.
- No tokens/passwords are stored or logged unsafely.
- Existing Phase 1.5 authorization continues to be the only authorization foundation.
- Automated tests pass and Playwright validates the main browser flows.
- No CRM domain behavior was duplicated into the auth frontend.

37. FINAL CONSOLE OUTPUT

At the end print a concise summary in this exact structure:

```
PHASE 2.5A - AUTH FRONTEND COMPLETE

- Identity/session mechanism:
- Frontend routes implemented:
- Backend endpoints added/used:
- Registration mode:
- Invite flow:
- Tenant-resolution behavior:
- Security controls verified:
- Tests added:
- Build/test/E2E result:
- Config/migrations:
- Deferred items:
- Open risks:
- Ready for Phase 2.5B CRM Frontend: YES / NO
```

Implementation intent - short reference

The desired architecture after this phase is straightforward: React owns authentication UX and shell behavior; .NET owns credential/session validation and tenant membership; Phase 1.5 Access owns authorization. The browser is never the source of truth for tenant membership or business permission.

```
React Auth UI
  ↓
.NET 10 Identity / Session endpoints
  ↓
Authenticated Principal
  ↓
Tenant resolution
  ↓
Phase 1.5 Access / Authorization
  ↓
Protected application APIs
```

After this phase, Phase 2.5B can integrate the real CRM Opportunity screens without first solving identity, session expiry, protected routing, or tenant switching on every page.